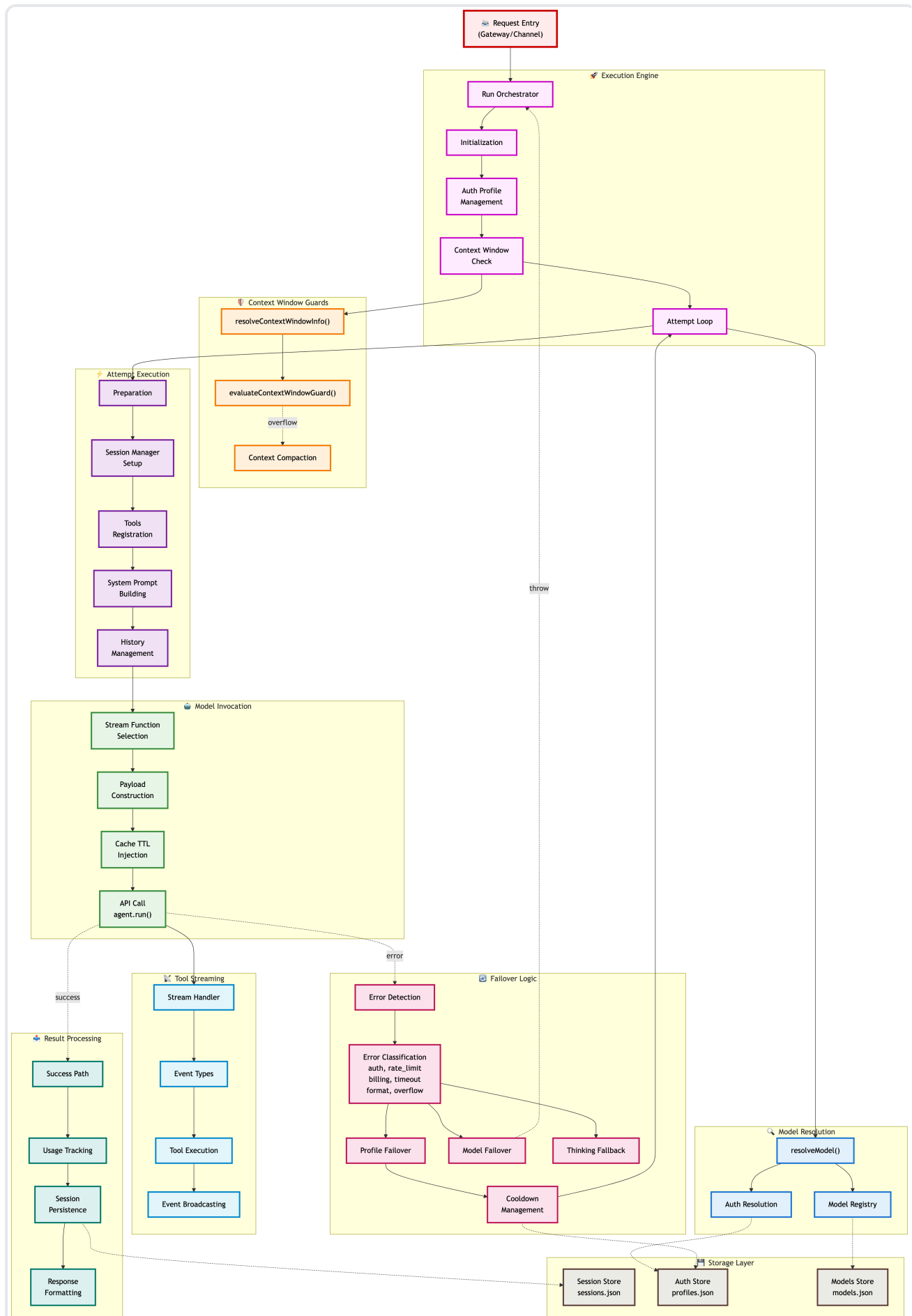


Clawdbot - Pi Execution Engine

Table of Contents

1. [Architecture Diagram](#)
2. [Overview](#)
3. [Execution Engine](#)
4. [Model Invocation](#)
5. [Tool Streaming](#)
6. [Failover Logic](#)
7. [Context Window Guards](#)
8. [Complete Execution Flow](#)
9. [Configuration Examples](#)

Pi Execution Engine Architecture



Overview

The Pi Execution Engine is the core runtime that executes AI agent sessions. Built on the Pi Agent SDK, it provides a robust, production-ready execution environment with advanced features:

- **Multi-provider Support:** Anthropic, OpenAI, Azure OpenAI, Google, GitHub Copilot, DeepSeek
- **Streaming Responses:** Real-time text and tool call streaming
- **Intelligent Failover:** Automatic retry with backup profiles and models
- **Context Management:** Automatic compaction and overflow handling
- **Tool Execution:** 50+ tools across 10+ categories

1. Execution Engine (Orchestrator)

Entry Point: `runEmbeddedPiAgent()` in `run.ts`

Initialization Phase

1. Resolve workspace directory
2. Load clawdbot configuration
3. Set up execution lanes (session + global queues)
4. Initialize abort controller
5. Load model registry

Auth Profile Management

Multiple auth profiles per provider for load balancing and failover:

```
Profile Order: 1. Locked profile (user-specified) 2. Preferred profile (from config) 3. All configured profiles (ordered) 4. undefined (anonymous/default)
Cooldown System: • Rate limit errors → 60s cooldown • Auth errors → 30s cooldown • Successful calls → Clear cooldown
```

Attempt Loop

```
for (profile in profileCandidates) { for (thinkingLevel in thinkingLevels) { try { result = await runEmbeddedAttempt(...) return result // Success! } catch (error) { if (isFailoverError(error)) { continue // Try next } throw error // Fatal } } }
```

Key Files: `src/agents/pi-embedded-runner/run.ts` , `src/agents/pi-embedded-runner/lanes.ts`

2. Model Invocation

Model Resolution

```
resolveModel(provider, modelId, agentDir, config) ↳ Load models.json  
registry ↳ Find model by provider/ID ↳ Load auth storage ↳ Return: {  
model, authStorage, modelRegistry }
```

Auth Resolution

Auth Modes:

- `api_key` : Direct API key
- `oauth` : OAuth access token
- `managedidentity` : Azure managed identity

Stream Function Selection

```
if (provider === "azureopenai") { streamFn = streamAzureOpenAINative } else  
{ streamFn = streamSimple }
```

Supported Providers

Provider	Models	Features
Anthropic	Claude 4.5/4.6	Extended thinking, prompt caching
OpenAI	GPT-4o, o1	Vision, function calling
Azure OpenAI	All OpenAI models	Managed identity, enterprise
Google	Gemini 2.0	Long context, multimodal
GitHub Copilot	Claude via GitHub	Enterprise auth
DeepSeek	DeepSeek models	Open source

Key Files: `src/agents/pi-embedded-runner/model.ts` , `src/agents/model-auth.ts`

3. Tool Streaming

Stream Handler Setup

```
subscribeEmbeddedPiSession(activeSession.agent, { onThinking: (data) => {  
  /* Extended thinking */ }, onText: (delta) => { /* Text streaming */ },  
  onToolCall: (tool) => { /* Tool invocation */ }, onToolResult: (result) =>  
  { /* Tool result */ }, onFinish: () => { /* Completion */ } })
```

Event Flow

Agent Event → Stream Handler → Gateway → WebSocket → Client UI

Tool Execution Flow

1. Agent requests tool call
2. Stream handler receives tool_call event
3. Parse tool name and parameters
4. Validate against registered tools
5. Execute tool (may be async)
6. Format result
7. Send tool_result back to agent
8. Agent continues with result

Key Files: `src/agents/pi-embedded-subscribe.ts`, `src/agents/pi-tools.ts`

4. Failover Logic

Error Classification

auth (401)

Invalid credentials

rate_limit (429)

Quota exceeded

billing (402)

Payment required

timeout (408)

Request timeout

format (400)

Invalid request

context_overflow

Context too large

Failover Hierarchy

Level 1: Auth Profile Failover

Error → Mark profile failed → Cooldown → Try next profile

Level 2: Thinking Level Fallback

Extended → Normal → Off

Level 3: Model Failover

Throw FailoverError → Gateway catches → Retry with fallback model

Cooldown Durations

Error Type	Cooldown	Reset
Rate limit	60 seconds	On success
Auth error	30 seconds	On success
Billing error	300 seconds	Manual
Timeout	10 seconds	On success

Retry Strategy

Max Attempts = profiles × thinkingLevels Example: 3 profiles × 2 levels = 6 attempts Attempt Order: 1. profile1 + extended 2. profile1 + normal 3. profile2 + extended 4. profile2 + normal 5. profile3 + extended 6. profile3 + normal

Key Files: `src/agents/failover-error.ts` , `src/agents/auth-profiles.ts`

5. Context Window Guards

Context Window Resolution

Priority Order: 1. model.contextWindow (from models.json) 2. config.models.providers[provider].models[].contextWindow 3. config.agents.defaults.contextTokens 4. DEFAULT_CONTEXT_TOKENS (128,000)

Guard Evaluation

Threshold	Tokens	Action
Hard minimum	16,000	Block execution
Warning	32,000	Log warning
Recommended	128,000+	Full functionality

Context Compaction

When context overflows, compact the session:

```
compactEmbeddedPiSessionDirect() ↳ Calculate available tokens ↳ Reserve  
for system, tools, response ↳ Trim oldest messages first ↳ Keep last N  
turns ↳ Preserve images when possible ↳ Save compacted session  
Compaction Strategy: System Prompt (10%) Tools (10%) Reserved (20%) History  
(50%) Response Buffer (10%)
```

Key Files: src/agents/context-window-guard.ts , src/agents/pi-embedded-runner/compact.ts

Complete Execution Flow

```
Request ↓ Initialize (workspace, config, lanes) ↓ Resolve Model  
(provider, model ID, auth) ↓ Check Context Window (warn/block if too  
small) ↓ FOR each auth profile: FOR each thinking level: ↓ Prepare  
Attempt (sandbox, tools, history) ↓ Build System Prompt ↓ Load History ↓  
Invoke Model (stream API call) ↓ Stream Events ↓ Execute Tools ↓ IF  
error: Classify → Failover Decision ↓ Try next profile/level or throw ↓  
IF success: Mark Profile Good ↓ Track Usage ↓ Save Session ↓ Return  
Response
```

Configuration Examples

Model Failover

```
{ "agents": { "defaults": { "model": { "primary": "anthropic/claude-sonnet-4.5", "fallbacks": [ "anthropic/claude-haiku-4.5", "openai/gpt-4o" ] } } }
```

Auth Profiles

```
{ "auth": { "profiles": { "anthropic:primary": { "provider": "anthropic", "mode": "api_key" }, "anthropic:backup": { "provider": "anthropic", "mode": "api_key" } } }
```

Context Window Override

```
{ "agents": { "defaults": { "contextTokens": 200000 } } }
```

Architecture Version: 1.0

Last Updated: 2026-02-08

Total Components: 5 major subsystems

Supported Providers: 6+ LLM providers